# Distributed Simulation Based Search

Jacob Hopkins, Shreeda Ghatpande, Sumaiya Azad, Fariba Amiri, and Andy Herbert

[1]

April 18, 2026

**ABSTRACT**

Modern multiplayer games must form matches that are fair, fast, and stable while operating under significant scale. Traditional matchmaking algorithms such as FIFO queues or rule-based matchers struggle to balance these competing requirements. This work investigates whether Approximate Nearest Neighbor (ANN) algorithms specifically FAISS can improve matchmaking quality and throughput in a distributed system. We develop a multi-shard matchmaking pipeline using FAISS for vector-based similarity search and compare it against FIFO and rule-based baselines. Our results show that weighted ANN embeddings significantly reduce match variance in skill and rank while maintaining high throughput and scalability.

## 1. Introduction

Matchmaking is a core component of modern multiplayer games, responsible for grouping players into balanced and playable matches. An ideal system should satisfy three properties:

1. Fairness players should face opponents of comparable skill and rank.
2. Speed wait time before entering a match should remain low.
3. Stability latency should be minimized, ideally by matching players within the same geographic region.

Traditional matchmaking systems often optimize one dimension at the cost of others. FIFO queues minimize wait time but ignore fairness; highly rule-dependent systems improve fairness but can cause long delays and high computational overhead. This research explores whether Approximate Nearest Neighbor search can serve as an efficient and scalable replacement for hand-crafted matchmaking logic. We propose a distributed matchmaking architecture where regional servers use FAISS to retrieve the most compatible players based on vectorized player attributes.

## 2. Related Work

Matchmaking has traditionally been dominated by methods such as:

- Skill-based systems (Elo, TrueSkill) that use rating models to pair players.
- Rule-based search trees (MatchTrees) that expand possible match sets under a hierarchy of constraints.
- Queue-based systems where wait time is the governing factor.

More recent work explores machine learning or embedding-based approaches, but few studies examine ANN methods in large-scale distributed matchmaking systems.

## 3. Synthetic dataset generation

For this project we opted to create our own synthetic dataset since a dataset that fits our needs is not readily available. For our dataset we tried to mimic a real world scenario as closely as possible. We designed it so each player has 6 attributes. A skill level, latency, region, rank, playtime hours and finally an ID.

The skill was generated using a Chi-square distribution (or a left leaning bell curve) to match distributions found commonly in online games where a majority of players are in the middle but a select few players are on the absolute ends of the skill curve. The Rank attribute was based on skill where players of different skills were placed into different rank 'buckets'. The reason for having both a rank and skill was to potentially represent more complex match making systems. This can help account for things like rank resets or players purposefully losing games to decrease their rank but our system didn't develop to the point where we tried dynamically adjusting attributes. Figure 1 shows the Chi-square distribution used to model skill attributes.

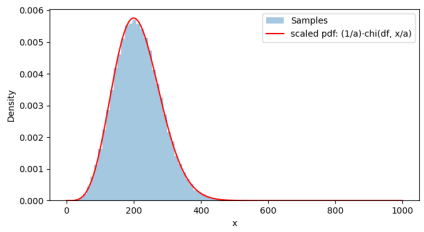


Fig. 1: Skill distribution

Latency attribute was generated using a chi-square distribution as well and used to represent a player's latency to their local regional server. As for the region attribute, players were put into one of six regions representing their local server. With those regions being either NA, EU, AS, SA, AF, OC. Then a graph was generated assigning these regions neighbors or next closest region. This is to measure how the algorithm performs when cross region matches occur. So games with NA and EU players were given a better score than players with SA and OC players since the regions do not neighbor.

Finally we had a playtime attribute chosen using a normal distribution which represents the players total playtime.

While we tried to accurately simulate a realistic dataset there still exists the downsides of using a synthetic dataset. First is our

dataset is not dynamic and does not account for trends over time. We overall weren't interested in measuring match making trends but it's still a real factor in match making. Second was the dataset did not account for hotzones or regions where players are more active during certain times of day.

## 4. Matchmaking Algorithms

We compare four matchmaking methods:

### 4.1. FIFO Queue Baseline

Players are matched strictly in arrival order. This system is trivial to implement but produces low-quality matches because it ignores player attributes.

### 4.2. FAISS (Unweighted ANN)

Each player is encoded as a vector:

- Skill
- Latency
- Rank Index
- Playtime
- One-hot region encoding

FAISS retrieves nearest neighbors based on Euclidean distance. This improves quality by considering attribute similarity but treats all attributes equally.

### 4.3. Weighted FAISS (Enhanced ANN Matchmaking)

Weighted FAISS extends the unweighted model by applying attribute-specific weights during vector construction. Attributes such as skill and latency, which have higher impact on perceived fairness, are scaled more strongly than secondary features like playtime. Formally, each feature xix_ixi is transformed to wixiw_i x_iwixi, where wiw_iwi is a tunable weight determined through iterative experimentation. This yields a weighted distance metric that prioritizes fairness-critical dimensions, improving match cohesion without sacrificing recall speed.

### 4.4. MatchTree Algorithm

In order to run different matchmaking methods, we devised an algorithm called MatchTree. MatchTree was designed to be an underlying algorithm for FIFO and FAISS algorithms, but due to time constraints MatchTree was used as another baseline. MatchTree is an algorithm that:

- Builds a tree of valid match combinations under ordered matchmaking methods (refered to as rule sets).
- Gradually relaxes constraints to minimize wait time.
- Performs priority-based match selection.

MatchTree is fast and scalable but can produce inconsistencies when players leave or are removed unless periodically rebuilt.

In order to find matchups for players in the queue, potential matchups must be determined, and "the best" one identified for players. Common algorithms for all of matchmaking exist, but focus on the formula for an ideal matchup, rather than the process for finding those matchups. In production video games, matchmaking often arises as a challenge of creating matches to create matches players will enjoy, while not making them wait too long.

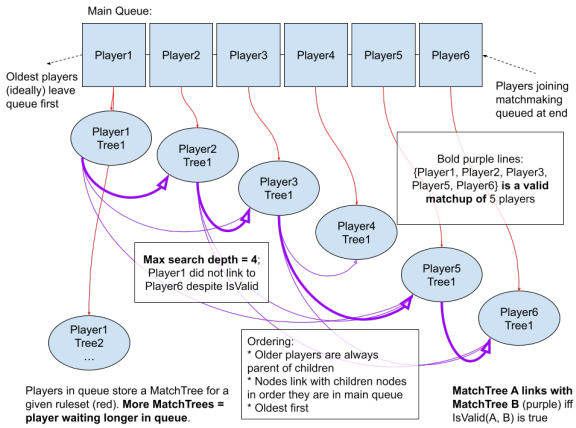


Fig. 2: Example instance of MatchTree searching for a match

MatchTree seeks to create potential matchups efficiently, prioritizing ideal matchups and reducing waiting time. MatchTree uses rulesets, which are a set of functions which return true- IsValid, if players can be in a match, and IsMatch, if a group of players forms a valid match (matching the required number of players). Rulesets have an additional priority function, which returns a value representing how ideal a matchup is. Rulesets also have a timeout.

Players join a queue FIFO-style. Each player stores a tree, which contains a ruleset, and links to newer players also in the queue, and a timer initially set to the player's current tree's ruleset. The timer ticks on frequency to update the trees, and if it expires, a new tree with a lower priority ruleset is made for that player. Lower priority rulesets are intended to allow less ideal matchups, down to baseline any-match, so that players who have been waiting longer have increased chances of being placed into a match, even if not ideal. Players link with other players up to a maximum number of links.

To identify valid matchups, a depth-first search is engaged, starting with a player's current tree (and any additional trees), in the queue. For a given path in the tree, if all players in the path are valid (function IsValid is true), iteration continues; if the players form a match (IsMatch is true), the group of players is added to the returned list. Iteration stops after a maximum number of matches has been found for a root, or if IsValid returns false.

MatchTree has various pruning and optimizations to speed up the search- a maximum number of links per tree node, and a max matches to return, which limit the total potential matchups and searches; and when players are added or removed, only a subset of queued players' trees update, rather than recomputing all player's trees. These optimizations slightly increase algorithm complexity and reduce the potential valid matchups, however allow the algorithm to run in a practical sense quickly, with linear-time or n(log)n complexity. There was an issue of inconsistency with the implementation of the algorithm regarding removing players from queue, so an additional tick is set to recompute a player's entire set of trees, sort of like a keyframe for video compression.

A paper dedicated to MatchTree is in development, intended on robustly defining the algorithm and providing testing metrics beyond what's presented in this report.

## 5. System Design

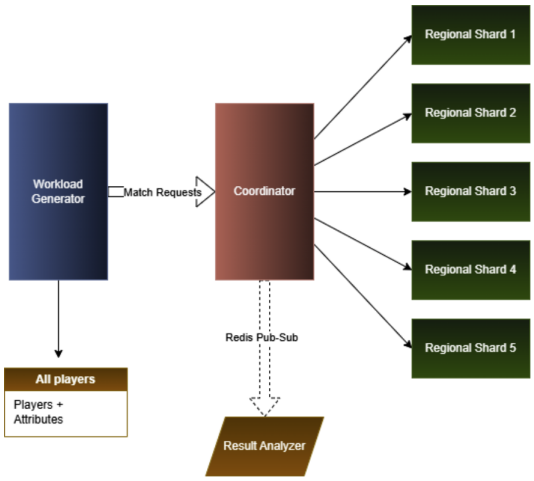


Fig. 3: System Design

We design a distributed matchmaking architecture consisting of four components:

### 5.1. Coordinator

The coordinator receives match requests from the workload generator and distributes them to regional server shards. It collects candidate players from shards and assembles final matches by applying FIFO, FAISS, or Weighted FAISS strategies.

### 5.2. Region Server Shards

Each shard corresponds to a geographic region (NA, EU, AS, SA, AF, OC). Shards:

- Load all regional players
- Convert them to numerical vectors
- Build a FAISS similarity Index
- Track online/offline and busy players
- provide gRPC APIs for:
  - RequestPlayers query nearest neighbors
  - ConfirmToMatch lock players once selected

Shards also simulate dynamic player availability through periodic reshuffling of online players.

On top of that, we have added two phase lock and two phase commit to ensure that if a local server has insufficient matches then neighboring regional servers are requested for available players. The RPCs that facilitate this design choice we have added two HoldMatchPlayers and ReleaseMatchPlayers

**HoldMatchPlayers**
HoldMatchPlayers(player_ids: repeated uint64) → HoldResponse

This RPC places a temporary hold on a set of player IDs. The hold prevents other match requests from selecting or modifying these players while the coordinator finalizes the match. Holds automatically expire after a short TTL to avoid deadlocks if the coordinator fails mid-process.
Behavior:

- Returns success only if all requested players are currently available.
- If any player is busy, offline, or already held, the call fails and no holds are applied.
- Optionally provides hold tokens used for later release.

Use case:
When the coordinator is assembling multi-region or cross-shard matches, it can hold candidate players before confirming the final match to avoid race conditions between shards.

**ReleaseMatchPlayers**
ReleaseMatchPlayers(player_ids: repeated uint64) → ReleaseResponse

Releases a previous hold or confirmed match lock on players. This RPC is called when:

- A match attempt fails and players need to be returned to the pool
- The coordinator times out a match sequence and wants to free locked players.
- A match is fully committed and the players should transition from “held” to “in-match” or “busy” state.

Behavior:

- Removes any hold or temporary lock associated with the listed players.
- If a player is not held, the shard simply ignores that ID.
- Does not affect long-term status like offline/online transitions.

### 5.3. Combined Use in Match Flow (example)

1. Coordinator calls RequestPlayers to get top-N ANN candidates.
2. The coordinator calls HoldMatchPlayers on the chosen set.
3. If hold succeeds:
   - Coordinator finalizes match compostion
   - Calls ConfirmToMatch to lock palyers for the match
4. If hold fails:
   - Corrdinator retries with another candidate set
5. After success or failure:
   - Coordinator calls ReleaseMatchPlayers for any players not used

This two-phase hold → confirm mechanism reduces race conditions in distributed matchmaking and prevents multiple match attempts from contending for the same players.

Below is a diagram depicting our server RPC architecture.

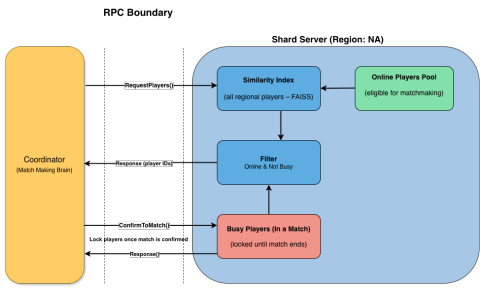


Fig. 4: System Diagram

### 5.4. Workload Generator

To simulate realistic matchmaking traffic, workload generation uses weighted sampling so the probability of choosing a player to base the FAISS search off of was equal to

(players_playtime _hours + 1)^hot_player_bias.

This hot player bias allows for players with more hours to queue more frequently. This yields a mix of casual and heavy users that resembles real-world matchmaking loads.

### 5.5. Result Analyzer

In order to capture match quality results we built a pubsub service that listens to matches set out by the coordinator and evaluates the quality of each match. In order to quantify the quality of each match we used an average weighted z-score. A z-score was chosen in order to normalize results as well as account for outliers such as high skill players.

When a match is sent to the match quality service it calculates a z-score for each attribute for each playing in the match. This z-score measures the standard deviation from the mean value for each attribute. After each attribute is scored we apply our attribute weight to give more value to attributes that are more valued. Then each attribute score can be summed and averaged to give each match a total z-score.

We then weighed the z-scores against the FIFO baseline. Below are some match quality results of the different algorithms. The blue represents the FIFO algorithm, the red represents the unweighted similarity search and the yellow represents the weighted similarity search. The lower the z-score the less deviation there is matchups.

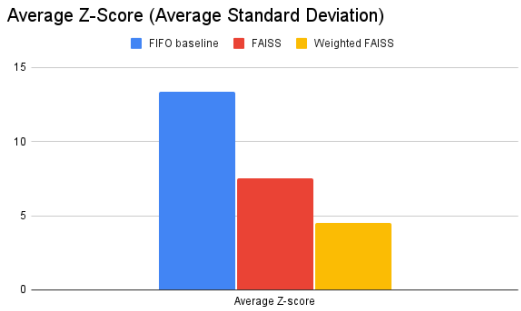


Fig. 5: Average Z-Score

## 6. Conclusions

Our research demonstrates that Approximate Nearest Neighbor methods can substantially improve matchmaking fairness while maintaining high throughput and scalability. Weighted FAISS embeddings significantly reduce variance in key attributes such as skill, rank, and latency compared to both FIFO and rule-based baselines.

These results suggest that ANN-based similarity search is a promising direction for next-generation matchmaking systems. Future work should evaluate ANN matchmaking under more dynamic conditions, including time-varying populations, player clustering, and real-world behavioral patterns.

## 7. Team Member Contribution

Shreeda Ghatpande - System architecture, FAISS search and Coordinator
Fariba Amiri - Server Shards and system design
Jacob Hopkins - Results Analyzer pub-sub, Server shard and benchmarking
Sumaiya Azad - Workload generation, coordinator, hot player bias, and benchmarking system
Andy Herbert - MatchTree algorithm and system design
All contributed to the presentation and the paper

## References

[1] J. Wang et al., "A Survey on Approximate Nearest Neighbor Search," IEEE TPAMI, 2020.
[2] K. Pelechrinis and D. Lo, "Online Matchmaking in Multiplayer Games: Modeling and Analysis," IEEE Transactions on Games, 2017.
[3] J. Johnson, M. Douze, and H. Jégou, "FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors," Facebook AI Research, 2017.
[4] R. Herbrich, T. Minka, and T. Graepel, "TrueSkill: A Bayesian Skill Rating System," NIPS, 2007.